code-ए-manipal 2.0

FINAL PROBLEM STATEMENTS

CODE-E-MANIPAL 2.0 | FINAL SHORTLIST

SOFTWARE + HARDWARE

BUILD-FOCUSED CHALLENGES

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 01 / 35

BUILD TYPE | SOFTWARE

ArcSense: Early Warning for Unstable Weld Signatures

PROBLEM STATEMENT

Welding quality can drift before a visible defect appears. Electrical signals and acoustic emissions contain clues about process stability, but normal changes in settings, material and sensor placement can resemble a fault. Build a monitoring aid that recognises changing process signatures and distinguishes likely instability from a changed measurement setup.

DESCRIPTION / IMPLEMENTATION

- Synchronise and segment current/voltage traces with acoustic events or use a supplied replay dataset.
- Extract time-frequency and sequence features; establish a rule-based baseline before training a compact model.
- Track uncertainty and an out-of-range state for unseen materials or operating settings.

DELIVERABLES

- Replay a labelled set of stable and unstable process windows with timeline and signal visualisation.
- Show a predicted state, evidence window, confidence and comparison with the baseline.
- Include a sensor-shift or changed-setting test. Real welding equipment is not required.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 02 / 35

BUILD TYPE | SOFTWARE

CivicSignal: Grouping Local-Issue Reports Without Losing Evidence

PROBLEM STATEMENT

A single road hazard, broken water line or streetlight failure may be reported many times in different languages and with different place names. Duplicate handling can reduce workload, but over-aggressive merging may hide a distinct incident or erase local context. Build a triage tool that groups likely duplicates while keeping the original evidence visible and preserving reports that may refer to separate events.

DESCRIPTION / IMPLEMENTATION

- Normalise spelling, transliteration, local place aliases, timestamps and approximate locations.
- Combine text similarity with time and geography to form candidate incident clusters.
- Keep "duplicate", "related" and "uncertain" as distinct outcomes, each with evidence and a human-review path.

DELIVERABLES

- Import synthetic or consented reports in at least two declared language/script conditions.
- Display clusters on a map or timeline, original snippets, confidence and merge/undo controls.
- Include a baseline using text similarity alone.

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 03 / 35

BUILD TYPE | SOFTWARE

SolveWatch: Forecasting Numerical Solver Failure from Run Traces

PROBLEM STATEMENT

Engineering and scientific simulations may consume substantial compute before diverging or producing unusable results. Residual values, convergence rates and parameter settings can offer early warning, but slow convergence is not always failure. Build a run monitor that estimates whether a simulation is likely to fail, explains the warning and avoids interrupting valid long-running experiments.

DESCRIPTION / IMPLEMENTATION

- Extract residual trajectories, iteration statistics and run configuration from supplied or generated traces.
- Compare a rule-based residual threshold with a learned or statistical early-warning model.
- Keep failure prediction separate from any automatic change to solver parameters.

DELIVERABLES

- A run viewer with residual curves, warning timeline and the features behind each alert.
- A replay engine for successful, slowly converging and divergent runs.
- A report that identifies warning lead time and cases where the system was uncertain.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 04 / 35

BUILD TYPE | SOFTWARE / GEOSPATIAL

RoadWeave: Rebuilding Connected Street Networks from Satellite Views

PROBLEM STATEMENT

Satellite images of urban areas often show roads interrupted by tree cover, building shadows, vehicles or clouds. A pixel-level road mask may therefore contain gaps that make route planning and infrastructure analysis unreliable. Build a system that reconstructs plausible road connectivity from imagery and identifies the links or intersections whose loss would most affect movement across a city.

DESCRIPTION / IMPLEMENTATION

- Segment roads from a small set of satellite image tiles, using a supplied model, a trained model or a documented baseline.
- Convert the predicted mask into a road graph and bridge likely gaps using distance, direction and network context; mark uncertain connections rather than inventing certainty.
- Rank key links using graph measures and simulate closures to estimate detours and changes in network accessibility.

DELIVERABLES

- A map view showing the source image, road mask, reconstructed graph and uncertain links.
- A bottleneck view with before/after routes for at least two simulated road closures.
- A runnable pipeline and a concise comparison of connectivity and route results on a small labelled or synthetic test set.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 05 / 35

BUILD TYPE | SOFTWARE / GEOSPATIAL

FloodMask: Flood Extent from Radar, With Honest Uncertainty

PROBLEM STATEMENT

Clouds often hide floods from optical satellites, so radar is used instead. Radar water maps can still confuse flood water with permanent water, radar shadow and urban areas. Build a mapper that outputs flood extent alongside a per-pixel "unsure" layer, rather than forcing a confident label everywhere.

DESCRIPTION / IMPLEMENTATION

- Data: Use Sen1Floods11, which includes Sentinel-1 imagery and 446 hand-labelled chips drawn from 11 flood events. Organizers must publish the permitted training data and reserve a genuinely organizer-controlled event/case set with labels withheld for final scoring; a publicly downloadable label set alone is not a hidden test.
- Implementation: Predict flood water, permanent water and not-water. Compare against the dataset Otsu-threshold radar baseline; weakly labelled data may be used if its noise is documented.
- Evaluation: Hold out whole flood events, not random chips. Report flood-class F1 and IoU versus Otsu, calibration, error-versus-coverage when the model abstains, and failures in urban areas, radar shadow and permanent water.
- Stretch: Mask aligned Sentinel-2 chips to simulate cloud and show where radar adds information. Explain mismatches and alignment limitations rather than treating every difference as a model error.

DELIVERABLES

- Minimum prototype: A map viewer with radar chip, flood prediction, per-pixel uncertainty layer, difference from the Otsu baseline and a per-event score table.
- A repeatable pipeline and report containing hidden-event scores, calibration and error-versus-coverage results.
- A short limitations note explaining the small hand-labelled set, event-level holdout and weak-label noise.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 06 / 35

BUILD TYPE | HARDWARE + SOFTWARE

FieldSense: Calibrating Soil-Moisture Readings Against Satellite Advisories

PROBLEM STATEMENT

Satellite-based crop stress estimates cover large areas but can miss local differences in soil, shade, irrigation and field conditions. Build a low-cost field sensing node that measures soil moisture and local conditions, checks sensor reliability, and compares readings with a satellite-derived or simulated irrigation advisory for a small test plot.

DESCRIPTION / IMPLEMENTATION

- Assemble a sensor node using a soil-moisture probe and a microcontroller; log readings with timestamps and basic sensor-health status.
- Calibrate readings against several known moisture conditions and identify drift, disconnected probes or implausible values.
- Display field readings beside a supplied or simulated crop-stress advisory, explaining where local measurements agree or disagree.

DELIVERABLES

- A working sensor prototype with live readings and a dashboard or local display.
- A calibration record and alerts for missing, stuck or out-of-range measurements.
- A field-versus-advisory comparison using a small repeatable test, with limitations clearly stated.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 07 / 35

BUILD TYPE | SOFTWARE + OPTIONAL DRONE HARDWARE

Monsoon Culvert Risk Triage from Drone Frames and Rainfall Histories

PROBLEM STATEMENT

Drainage crossings can become blocked by sediment, vegetation or waste shortly before intense rain. Inspection teams cannot revisit every culvert, and a visually clear image does not guarantee hydraulic capacity. Rank sites for inspection using visual evidence, catchment context and rainfall history.

DESCRIPTION / IMPLEMENTATION

- Implementation challenge: Combine change detection over image dates with a catchment/road graph, rainfall accumulation, terrain and prior incident evidence. Represent missing imagery and uncertain blockage labels explicitly; distinguish observed blockage from inferred risk.
- Evaluation / failure tests: Evaluate top-k inspection yield on held-out annotated or synthetic cases, calibration of risk scores, route cost and sensitivity to missing or stale reports. Include a simple rainfall-only baseline.
- Stretch / differentiator: Inject a suddenly blocked culvert, false drone observation and corrupted rainfall feed. Explain which sites rise or fall in priority and why.
- Suggested public-sector relevance: State disaster-management authorities, PWD and municipal engineering cells.

DELIVERABLES

- Minimum acceptable prototype: Deliver a geospatial triage map, suspected blockage masks or annotations, a ranked inspection queue, confidence ranges and a route suggestion. Each ranking must show which evidence contributed and what is missing.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 01 | AI/ML & EARTH OBSERVATION

PS 08 / 35

BUILD TYPE | SOFTWARE + OPTIONAL EDGE HARDWARE

Perception That Knows When It Cannot See: Edge AI Under Dust, Glare and Smoke

PROBLEM STATEMENT

Field vision models can be confidently wrong under dust, glare, smoke, low light or partial occlusion. In inspection and field robotics, the system should know when it lacks sufficient evidence and choose to abstain or request another view.

DESCRIPTION / IMPLEMENTATION

- Implementation challenge: Build a corruption benchmark plus a selective-prediction layer. Compare confidence calibration, out-of-distribution scoring and abstention policies under CPU/RAM/latency constraints. Separate image corruption from genuinely unfamiliar content.
- Evaluation / failure tests: Report risk-versus-coverage curves, calibration error, false-confidence rate, latency and memory use. Include mixed corruptions and unseen test conditions, not only isolated filters.
- Stretch / differentiator: Add a limited re-calibration budget and test whether active selection of a few reviewed examples improves reliability without exceeding edge constraints.
- Suggested public-sector relevance: DRDO-aligned field-research inspiration, industrial inspection and rail maintenance. The brief does not imply access to defence data.

DELIVERABLES

- Minimum acceptable prototype: Deliver a baseline and robust model, a confidence/coverage display, a failure-cluster explorer and a measured laptop or edge-device profile. For each input, show prediction, confidence, abstain/escalate decision and reason.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 02 | HEALTHTECH

PS 09 / 35

BUILD TYPE | SOFTWARE / SIMULATION

CrewSignal: Personal Health Trend Support for Long-Duration Missions

PROBLEM STATEMENT

During long missions, crew members may experience changing sleep, activity, cardiovascular indicators, stress and environmental exposures while communication with medical support is delayed. A fixed population threshold may miss a meaningful change for one person or raise repeated alarms for another. Build an offline-first monitoring and decision-support prototype that tracks personal trends and makes uncertainty explicit.

DESCRIPTION / IMPLEMENTATION

- Combine simulated heart rate, blood pressure, oxygen saturation, activity, sleep, self-reported symptoms and available habitat measurements.
- Establish personal baselines and detect persistent deviations or combinations of moderate changes.
- Separate an observed measurement, a suspected concern and a confirmed diagnosis; the system must not diagnose or prescribe treatment.
- Continue recording and alerting without connectivity, then reconcile delayed or conflicting records.

DELIVERABLES

- An astronaut profile, mission timeline, simulated sensor stream and symptom-log interface.
- Trend charts, anomaly flags, evidence behind each flag, uncertainty and an explicit insufficient-evidence state.
- A decision-support workflow for repeating a measurement, logging a symptom or escalating through an approved protocol.
- Scenario replay and local storage with timestamps. Synthetic data only.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 02 | HEALTHTECH

PS 10 / 35

BUILD TYPE | HARDWARE + SOFTWARE

OpenForce: Measuring the Effort to Open Everyday Packaging

PROBLEM STATEMENT

"Hard to open" is a feeling until someone measures it. Build an instrumented rig that measures the peak force or torque and effort profile needed to open packaging, so opening aids can be compared with repeatable measurements rather than impressions.

DESCRIPTION / IMPLEMENTATION

- Implementation: Build a rig with a load cell or torque sensor, amplifier and microcontroller. Calibrate it against known masses or known torque, and log timestamped effort curves.
- Test at least three types of empty packaging with and without an opening aid, keeping the mounting and test procedure consistent. The no-aid result is the baseline.
- Evaluation: Report calibration error, repeatability across repeated trials, drift over time and measurement uncertainty. Conduct a blind comparison of two aid designs using organizer-selected samples or test conditions; disclose the test protocol after scoring.
- Stretch: Add a second measurement axis, such as grip force versus twist torque. Use empty packaging only; make no clinical claims. Human-subject trials are optional and require informed consent.

DELIVERABLES

- Minimum prototype: A working rig with live readout, sensor calibration record and timestamped force/torque curves.
- A results table comparing at least three packaging types with and without an aid, including repeated-trial error bars and a blind aid comparison.
- A short test protocol and limitations note describing measurement range, uncertainty and sources of drift.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 02 | HEALTHTECH

PS 11 / 35

BUILD TYPE | SOFTWARE / SIMULATION

BloodStock: Time-Aware Allocation of Perishable Blood Components

PROBLEM STATEMENT

A network of hospitals may have blood components with different expiry times, compatibility constraints, delivery durations and reservations. A unit available nearby may not be suitable for a particular request, while another unit may expire before it can arrive. Build a planning tool that identifies feasible transfers and explains trade-offs without making clinical decisions.

DESCRIPTION / IMPLEMENTATION

- Model stock, compatibility rules, expiry, transport times and urgent requests across several fictional facilities.
- Use a constraint solver or min-cost flow model to recommend reservations and transfers.
- Prevent double allocation and preserve hard compatibility and expiry constraints during replanning.

DELIVERABLES

- A synthetic multi-hospital inventory simulator with a simple nearest-available baseline.
- A recommendation list showing source, destination, feasibility, estimated arrival and reason.
- Scenario controls for transport failure, demand spikes, cancellations and new stock arrivals.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 02 | HEALTHTECH

PS 12 / 35

BUILD TYPE | SOFTWARE / OPTIONAL HARDWARE

BreathGate: Breathing-Rate Estimation That Abstains When the Signal Is Bad

PROBLEM STATEMENT

Breathing rate can be estimated from a pulse-sensor signal, but noise and motion can make estimates unreliable. Build an estimator that reports a rate only when it has enough signal evidence and otherwise says "unreliable". It must not diagnose or recommend treatment.

DESCRIPTION / IMPLEMENTATION

- Data: Use the BIDMC dataset of 53 eight-minute recordings containing photoplethysmography (PPG), impedance respiration and ECG signals with reference respiratory rates. The two manual breath annotations differ slightly; report their spread as part of the reference uncertainty.
- Implementation: Add signal-quality gating and a breathing-rate estimator; compare it with a simple spectral-peak baseline. Organizers must use a subject-level split and withhold evaluation references in a locked scoring set; publicly downloadable labels alone cannot be described as hidden test labels.
- Evaluation: Split by subject, not by windows. Report error in breaths per minute against coverage, errors on accepted and rejected windows, and results after injecting motion noise and signal dropouts.
- Stretch: Run on a microcontroller-class device and report latency and memory use. Clearly label the result as a signal-processing prototype, not a medical device.

DELIVERABLES

- Minimum prototype: A signal viewer showing the waveform, estimated rate, confidence or quality indicator, and an explicit abstain flag.
- An error-versus-coverage curve and a report comparing the spectral-peak baseline with the proposed estimator on held-out subjects.
- A failure-case summary for motion noise, dropouts and disagreement between reference annotations.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 03 | FINTECH / EDTECH / LEGALTECH

PS 13 / 35

BUILD TYPE | SOFTWARE

MilestonePay: Evidence-Gated Release for Staged Payments

PROBLEM STATEMENT

Staged payments between a client and a service provider can become disputed when acceptance evidence is incomplete, contradictory or submitted late. A release workflow must show which requirements have been met, identify missing evidence and allow a dispute to be escalated. Build a simulated payment-control console; no real money or live payment rails are involved.

DESCRIPTION / IMPLEMENTATION

- Represent milestones as explicit conditions with evidence requirements, deadlines and authorised reviewers.
- Validate submitted files, approvals and timestamps while retaining an immutable or tamper-evident event history.
- Distinguish ready-for-review, approved, rejected and disputed states; do not infer approval from missing records.

DELIVERABLES

- A fictional project with several payment milestones and a staged evidence upload.
- A rules-based release gate, reviewer queue and auditable state-transition history.
- A dispute workflow showing the exact condition or evidence item that blocks release.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 03 | FINTECH / EDTECH / LEGALTECH

PS 14 / 35

BUILD TYPE | SOFTWARE

LabTrail: Verifiable Provenance for Collaborative Practical Work

PROBLEM STATEMENT

A practical project often involves shared experiments, evolving code, raw measurement files and multiple contributors. When notes or results are edited or synchronised late, it can be difficult to reconstruct which artifact supported a conclusion and who changed it. Build a lab notebook that preserves a clear, reviewable history without relying on invasive student surveillance.

DESCRIPTION / IMPLEMENTATION

- Link experiment notes to datasets, code versions, file hashes and named contributors.
- Record changes as an append-only or tamper-evident history and reconcile delayed offline updates.
- Separate evidence of a contribution from an automatic judgement of academic merit.

DELIVERABLES

- Create and edit an experiment record with attached artifacts, timestamps and version history.
- Verify artifact integrity and display the chain from a claim to the relevant raw file or run.
- Support offline changes, duplicate submissions and a reviewer export.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 03 | FINTECH / EDTECH / LEGALTECH

PS 15 / 35

BUILD TYPE | SOFTWARE

CaseBundle: Evidence-Complete Packs for Tenancy Disputes

PROBLEM STATEMENT

A tenancy dispute can involve notices, messages, receipts, photographs and dated records from multiple people. Preparing a clear evidence pack is time-consuming, while missed sensitive details or overly broad redactions can create additional problems. Build a document-preparation aid for a bounded fictional case that organises evidence and reveals gaps without giving legal advice or deciding the dispute.

DESCRIPTION / IMPLEMENTATION

- Extract dates, parties, events and document references from synthetic case files.
- Create an event timeline and evidence index with exact source locations for each extracted item.
- Apply reversible, human-approved redactions and flag missing, inconsistent or duplicate records.

DELIVERABLES

- Upload a small set of fictional PDFs and images, then create a timeline and indexed evidence pack.
- Show source page/paragraph for each extracted fact and maintain a redaction audit trail.
- Use a single declared jurisdiction checklist supplied for the exercise, with unknown or unsupported points flagged for review.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 03 | FINTECH / EDTECH / LEGALTECH

PS 16 / 35

BUILD TYPE | SOFTWARE ANALYTICS WORKBENCH

UPI Fraud-Ring Graph Investigator with Explainable Alerts

PROBLEM STATEMENT

Fraud analysts may receive fragmented complaints involving collect requests, mule accounts, repeated device signals and rapid onward transfers. The hard problem is to surface coordinated behaviour without treating a suspicious pattern as proof or flagging every shared device as fraud.

DESCRIPTION / IMPLEMENTATION

- Implementation challenge: Build a temporal heterogeneous graph across synthetic accounts, devices, merchants, complaints and transfers. Combine transparent rules with temporal motifs or graph clustering, and retain a traceable evidence path for each alert.
- Evaluation / failure tests: Report precision/recall at a fixed analyst alert budget, false-positive rate on benign shared-device cases and stability when events arrive late or out of order. Keep decisions human-reviewed.
- Stretch / differentiator: Add a counterfactual panel showing which edges or events would change the alert and what benign explanation could fit the same pattern.
- Suggested public-sector relevance: Public-sector banks, authorised fraud-analysis research and cybercrime prevention workflows. Never connect to live UPI rails or use personal financial data.

DELIVERABLES

- Minimum acceptable prototype: Deliver a graph explorer, analyst queue, evidence path, baseline comparison and labelled synthetic benchmark. Include class imbalance, benign shared devices, incomplete records and delayed complaint reporting.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 03 | FINTECH / EDTECH / LEGALTECH

PS 17 / 35

BUILD TYPE | SOFTWARE + OPTIONAL LOCAL HARDWARE AGENT

Offline Practical Examination Integrity with Signed Lab Evidence

PROBLEM STATEMENT

Engineering practical exams may run in labs with limited connectivity. A final score alone cannot explain a disputed run, while central logging may fail during outages. Build evidence that remains verifiable without invasive surveillance or continuous internet access.

DESCRIPTION / IMPLEMENTATION

- Implementation challenge: Define a threat model, sign event batches, hash inputs/artifacts, chain event records, and implement deterministic reconciliation for duplicate, delayed or conflicting sync. Explicitly handle workstation identity, clock drift and recovery after power loss.
- Evaluation / failure tests: Test log deletion, rollback, clock manipulation, replay, workstation swap, interrupted power and sync conflicts. Report tamper detection and false alarms during legitimate offline operation.
- Stretch / differentiator: Add a two-party verification workflow in which a student can receive a receipt while the assessor can validate the same evidence independently.
- Suggested public-sector relevance: Public technical institutes, examination bodies and vocational training labs.

DELIVERABLES

- Minimum acceptable prototype: Deliver a local exam runner or simulated lab, a signed event manifest, artifact hashes, a verifier tool and a tamper demonstration. Exclude biometric monitoring and covert camera surveillance.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 04 | CYBERSECURITY

PS 18 / 35

BUILD TYPE | SOFTWARE

AccessLease: Finding Temporary Permissions That Never Expired

PROBLEM STATEMENT

Institutions grant temporary access to shared systems for projects, events, maintenance and visiting staff. Permissions can remain active after the work ends, or lack a clear owner and justification. Build an audit tool that detects these cases, explains the evidence and proposes a review action without connecting to real institutional accounts.

DESCRIPTION / IMPLEMENTATION

- Ingest a mock identity, role, resource and delegation dataset with owners, expiry times and purpose.
- Compare effective permissions with a supplied policy and identify stale, over-broad or ownerless grants.
- Distinguish a confirmed policy violation from missing metadata that requires review.

DELIVERABLES

- A permission graph or table with a priority-ordered review queue.
- A policy explanation for each finding and a simulated expiry/revocation workflow.
- An audit log recording reviewer decisions and changes.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 04 | CYBERSECURITY

PS 19 / 35

BUILD TYPE | SOFTWARE / SIMULATION

RestoreGraph: Safe Recovery Ordering for Dependent Services

PROBLEM STATEMENT

After an outage, restoring individual services in the wrong order can cause repeated failures when databases, identity systems, queues or storage dependencies are not ready. A backup inventory alone cannot explain which restoration sequence is safe. Build a verifier that derives a recovery order, identifies unavailable prerequisites and shows when no safe plan can be formed.

DESCRIPTION / IMPLEMENTATION

- Represent services, data stores, dependencies, recovery-point objectives and backup status as a graph.
- Compute a valid restore sequence and detect cycles, stale backups and missing prerequisites.
- Simulate partial restoration and verify readiness before dependent services are brought online.

DELIVERABLES

- A service-dependency editor or importer and a generated recovery plan.
- A simulated execution timeline with readiness checks, blocked steps and evidence for each dependency.
- A baseline that restores services in a fixed or naive order for comparison.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 04 | CYBERSECURITY

PS 20 / 35

BUILD TYPE | SOFTWARE

TraceBack: Reconstructing an Intrusion Timeline from Raw Logs

PROBLEM STATEMENT

After an incident, analysts face thousands of unlabelled events from many machines. Build a tool that proposes a timeline of attacker steps, links each step to the log evidence behind it, and marks weak links as "uncertain". The tool works on logs only and must not include exploit code.

DESCRIPTION / IMPLEMENTATION

- Data: Start with OTRF Security Datasets (formerly Mordor), which contain recorded events from simulated adversary techniques alongside benign activity. Because public traces may be too small for multi-host evaluation, organizers must also prepare a reproducible multi-host scenario with benign noise and keep its ground-truth attack-step list hidden until scoring.
- Implementation: Correlate events across hosts and users into candidate attack steps. Every proposed step must link to supporting log evidence, and weak causal links must be explicitly marked uncertain.
- Evaluation: Compare with the baseline that sorts events by time and groups them by host. Score recall of true attack steps, precision of causal links, false links caused by benign bursts, and robustness to clock skew and late-arriving logs.
- Stretch: Add a counterfactual panel showing which events would change a proposed link and what benign explanation could fit the same pattern.

DELIVERABLES

- Minimum prototype: A timeline view, evidence drill-down, uncertain-link flags and a score report against the organizer-held scenario.
- A repeatable log parser/correlation pipeline and a baseline comparison with attack-step recall, causal-link precision and false-link counts.
- A failure-case summary covering clock skew, delayed events and benign activity; no live network access or exploit code.

TRACK 05 | GENERATIVE AI & LLM SYSTEMS

PS 21 / 35

BUILD TYPE | SOFTWARE

ViewProof: CAD Review Answers Anchored to Geometry

PROBLEM STATEMENT

Engineering questions often refer to a hole, interface, clearance or assembly feature visible only in a particular drawing or model view. A text-only assistant may return a plausible answer that is not supported by the geometry. Build a review companion that answers from provided CAD views or annotations and points to the exact visual evidence; it must abstain when the relevant feature is not visible.

DESCRIPTION / IMPLEMENTATION

- Index model views, labels, dimensions and part identifiers.
- Retrieve a relevant view or annotation before composing an answer.
- Keep answers linked to evidence and distinguish visible facts from interpretation.

DELIVERABLES

- A small supplied assembly/drawing set and a query interface for bounded engineering questions.
- Answers with a highlighted view, feature or annotation as evidence.
- An "not visible / insufficient evidence" response for unsupported questions.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 05 | GENERATIVE AI & LLM SYSTEMS

PS 22 / 35

BUILD TYPE | SOFTWARE

DiagramCheck: Semantic Audit of AI-Generated Scientific Figures

PROBLEM STATEMENT

A generated scientific diagram can look polished while reversing an arrow, omitting a component, mislabelling a unit or contradicting the accompanying explanation. Build an auditor that checks a diagram against a supplied reference description and flags concrete inconsistencies instead of merely judging visual quality.

DESCRIPTION / IMPLEMENTATION

- Represent diagram entities, labels, links and directionality in a structured form.
- Compare them with a specification or accompanying explanatory text.
- Localise each mismatch and separate definite structural errors from uncertain semantic concerns.

DELIVERABLES

- Upload a diagram and its short reference specification.
- Return an annotated image, discrepancy list and links to the relevant text or diagram region.
- Include a simple checklist baseline and a set of intentionally corrupted diagrams.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 05 | GENERATIVE AI & LLM SYSTEMS

PS 23 / 35

BUILD TYPE | SOFTWARE

SonicAccess: Coverage Audit for Audio Descriptions in Learning Videos

PROBLEM STATEMENT

Recorded lessons and demonstrations often contain essential visual information-changes to a diagram, a highlighted control, a plotted trend-that may not be conveyed by narration or audio description. A quality checker should locate these gaps and align its findings to the video timeline, rather than generate unsupported descriptions of its own.

DESCRIPTION / IMPLEMENTATION

- Align video segments, transcript/audio-description text and notable visual events.
- Identify potentially important visual events without a corresponding description and flag descriptions that conflict with the scene.
- Show evidence and uncertainty so a content author can review each proposed gap.

DELIVERABLES

- Process a short supplied or synthetic lesson clip and its transcript/description track.
- Display a timeline of detected visual events, covered events and candidate gaps.
- Allow a reviewer to mark a finding correct/incorrect and export a revision checklist.

TRACK 06 | MULTI-AGENT SYSTEMS / LOGISTICS

PS 24 / 35

BUILD TYPE | SOFTWARE + OPTIONAL IOT SENSOR

ChillPath: Cold-Chain Excursion Recovery and Shipment Traceability

PROBLEM STATEMENT

Temperature-sensitive goods may experience an excursion because refrigeration weakens, transfers are delayed or a destination becomes unavailable. Managers need more than a threshold alarm: they need to know what was observed, whether the sensor is trustworthy, how much exposure time has accumulated and which intervention is feasible. The prototype must not decide that a product is safe to use.

DESCRIPTION / IMPLEMENTATION

- Track simulated or physical temperature/humidity readings against declared product-specific limits and exposure budgets.
- Link each reading to shipment, route, handoff, sensor status and timestamp; preserve data through connectivity loss.
- Rank interventions such as equipment inspection, delivery prioritisation or rerouting using remaining tolerance and time-to-destination.

DELIVERABLES

- Shipment management, live/replayed telemetry, route timeline and excursion-history views.
- Distinct states for valid excursion, suspected sensor fault and insufficient evidence.
- Offline storage, duplicate-safe synchronisation and scenario controls for refrigeration failure, traffic delay and destination closure.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 06 | MULTI-AGENT SYSTEMS / LOGISTICS

PS 25 / 35

BUILD TYPE | SOFTWARE / SIMULATION

RepairLoop: Negotiating the Return Path for Repairable Electronics

PROBLEM STATEMENT

Returned electronics may be repaired, refurbished for resale, used for parts or recycled. The best option depends on uncertain condition, parts availability, repair capacity, transport cost, warranty obligations and environmental impact. Independent actors can have competing incentives, so a single fixed rule may send recoverable products to waste or overload a repair depot.

DESCRIPTION / IMPLEMENTATION

- Represent the customer/return point, carrier, repair depot, parts supplier and refurbisher as agents with different costs and constraints.
- Negotiate routing and capacity while keeping product state, warranty and chain-of-custody records consistent.
- Compare the negotiated decision with a fixed-priority baseline and make the reason for each allocation inspectable.

DELIVERABLES

- A simulated return network with repair, parts, refurbishment and recycling paths.
- Agents that offer/accept/reject jobs under changing capacity and cost.
- A dashboard showing product disposition, negotiation trace and circularity metrics.

TRACK 06 | MULTI-AGENT SYSTEMS / LOGISTICS

PS 26 / 35

BUILD TYPE | HARDWARE + SOFTWARE

ShelfSense: Physical Stock-Change Detection for Shared Storage

PROBLEM STATEMENT

Shared labs and small stores often rely on manual stock checks, so items may be removed, returned to the wrong place or recorded late. Build an instrumented shelf or bin that detects stock changes and reconciles them with a simple inventory record.

DESCRIPTION / IMPLEMENTATION

- Use a load cell, break-beam sensor, RFID reader or another documented low-cost sensing method to detect item movement.
- Associate observations with a shelf/bin identifier and maintain a timestamped inventory event log; mark uncertain readings for manual review.
- Test repeated removals, returns, mixed item weights and sensor interruptions, and compare automated counts with a manually checked baseline.

DELIVERABLES

- A physical shelf/bin demonstrator with live stock-change detection.
- An inventory dashboard or local display showing counts, event history and uncertain changes.
- A repeatable test report showing count accuracy and how the system handles sensor failures.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 06 | MULTI-AGENT SYSTEMS / LOGISTICS

PS 27 / 35

BUILD TYPE | SOFTWARE SIMULATION + OPTIONAL MAP UI

Flood Rescue Dispatch Where Boats, Shelters and Roads Negotiate in Real Time

PROBLEM STATEMENT

Flood response plans become stale as routes close, shelters fill and reports conflict. Coordinate rescue teams, boats, shelters and incident reports as autonomous agents with partial information, resource contention and different urgency levels.

DESCRIPTION / IMPLEMENTATION

- Implementation challenge: Implement dynamic re-planning over a changing road/water graph, with uncertainty-tagged reports, capacity constraints, fairness criteria and explicit safety invariants. Compare decentralised negotiation with a centralised baseline.
- Evaluation / failure tests: Measure time to service, unmet requests, resource idle time, route violations, fairness across neighbourhoods and recovery after contradictory updates.
- Stretch / differentiator: Trigger simultaneous bridge closure, boat failure and shelter overflow; show which commitments are cancelled, who is re-assigned and why.
- Suggested public-sector relevance: Disaster-management authorities, fire and emergency services.

DELIVERABLES

- Minimum acceptable prototype: Deliver a time-stepped simulation, injected events, assignment log, reroute explanation and after-action replay. Use synthetic locations and scenarios rather than live operational feeds.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 07 | GAMING & IMMERSIVE TECH

PS 28 / 35

BUILD TYPE | SOFTWARE

ClueMesh: Mystery Generation with Verifiable Evidence Paths

PROBLEM STATEMENT

Procedurally generated mysteries can produce engaging dialogue but fail when clues contradict each other, a required fact is never discoverable or a branch makes the solution impossible. Build a mystery authoring engine where narrative variety is allowed, but every released case has a checkable evidence structure and at least one valid route to a justified conclusion.

DESCRIPTION / IMPLEMENTATION

- Represent suspects, events, clues, dependencies and conclusions as a directed evidence graph.
- Generate or assemble branches under rules that prevent unsupported conclusions and accidental contradictions.
- Run a verifier before a case becomes playable, then explain why rejected cases failed.

DELIVERABLES

- A playable mystery with multiple clue paths and an interactive evidence board.
- A graph view and automatic checks for missing prerequisites, contradiction and unreachable conclusions.
- A seed-based generator or editor that can make more than one test case.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 07 | GAMING & IMMERSIVE TECH

PS 29 / 35

BUILD TYPE | SOFTWARE

StealthGraph: Finding Sightline and Sound Leaks in Game Levels

PROBLEM STATEMENT

A stealth level may appear balanced to its designer while geometry creates unintended visibility through walls, sound travels farther than expected or patrol routes leave exploitable gaps. Build a level-analysis tool that discovers and visualises these problems before playtesting, using a declared simplified visibility and sound model rather than requiring a full commercial engine.

DESCRIPTION / IMPLEMENTATION

- Import or construct a grid-based level with walls, doors, materials, sound sources and patrol routes.
- Compute sightline exposure and sound propagation under a configurable simplified model.
- Identify suspicious player routes or guard blind spots and link each finding to the geometry that causes it.

DELIVERABLES

- A small level editor or importer with visibility and sound overlays.
- A report of exposed routes, acoustic leaks and blind spots.
- A baseline map with hand-authored known defects for verification.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 07 | GAMING & IMMERSIVE TECH

PS 30 / 35

BUILD TYPE | SOFTWARE

Landmark Quest: Spatial Puzzles Built for Low-Vision Play

PROBLEM STATEMENT

Spatial puzzle games often depend on colour, small text or rapid visual search, excluding some players. Build a game where orientation and progress can be understood through consistent audio cues, shape/texture differences and optional physical landmark tokens, while preserving meaningful challenge rather than reducing the experience to a quiz.

DESCRIPTION / IMPLEMENTATION

- Design a small spatial world with landmarks that can be identified by more than colour alone.
- Provide adjustable audio cues, text scaling, contrast and non-visual interaction without revealing every answer.
- Make navigation feedback reliable and test for ambiguous or inaccessible landmark combinations.

DELIVERABLES

- At least three puzzle rooms with a consistent landmark vocabulary and a clear goal.
- Audio/large-text/tactile-token alternatives for essential information.
- A settings panel and a test mode that simulates reduced visual cues.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 08 | SMART CITY & INFRASTRUCTURE

PS 31 / 35

BUILD TYPE | SOFTWARE

CoolRoute: Pedestrian Routing with Shade and Heat-Exposure Budgets

PROBLEM STATEMENT

The shortest walking route may expose a pedestrian to prolonged heat, unshaded crossings or long waits at exposed junctions. A useful route planner should account for shade, time of day, walking effort and accessibility, and should show when a cooler route adds too much distance or passes through uncertain data.

DESCRIPTION / IMPLEMENTATION

- Represent streets and crossings as a graph with segment length, shade estimate, slope/accessibility attributes and time-dependent heat exposure.
- Optimise a declared heat-exposure budget while preserving hard constraints such as step-free access when requested.
- Show where input data is missing and avoid presenting modelled heat as a precise personal medical risk.

DELIVERABLES

- A map or synthetic neighbourhood with at least two route choices.
- A baseline shortest-path route and an exposure-aware alternative with an explanation of trade-offs.
- Scenario controls for time of day, shade availability and a closed footpath.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 08 | SMART CITY & INFRASTRUCTURE

PS 32 / 35

BUILD TYPE | SOFTWARE / GEOSPATIAL

Subterra Clash: Detecting Conflicts in Shared Underground Works

PROBLEM STATEMENT

Water, sewer, power, telecom and planned construction projects often occupy overlapping underground corridors. Separate permit records may use different coordinate precision, clearance rules or planned dates, making a conflict hard to spot before excavation. Build a review tool that identifies probable spatial and scheduling conflicts, shows their evidence and leaves ambiguous cases for engineering review.

DESCRIPTION / IMPLEMENTATION

- Represent utility corridors and proposed works as 2D/3D geometries with depth, clearance, uncertainty and schedule.
- Detect geometric intersections and near-conflicts while accounting for declared tolerance and missing metadata.
- Group related clashes and propose a review order without automatically approving or rejecting a permit.

DELIVERABLES

- A small map/cross-section viewer with synthetic utility layers and proposed work corridors.
- A conflict list with location, affected assets, clearance measurement and source record.
- A severity-ranked review queue and exportable inspection report.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 08 | SMART CITY & INFRASTRUCTURE

PS 33 / 35

BUILD TYPE | HARDWARE + SOFTWARE

VentRate: Estimating Room Ventilation Rate from CO2, With Uncertainty

PROBLEM STATEMENT

A CO2 reading is not a ventilation rate. Build a sensor node that estimates how quickly a room's air is being replaced, reports an uncertainty range, and indicates when the estimate should not be trusted.

DESCRIPTION / IMPLEMENTATION

- Implementation: Build a CO2 sensor node with a microcontroller. State whether the sensor measures CO2 directly or infers it, and estimate air-change rate from a decay curve. Decay testing requires an unoccupied space and approximately constant ventilation during the test.
- Evaluation: Test at least three ventilation conditions against a documented reference, such as a fan with measured flow or defined door/window states. Compare the estimate against a plain ppm-threshold baseline.
- Evaluate sensitivity to sensor placement and non-uniform mixing; report reference-versus-estimate error and an uncertainty range. Organizers should reserve one condition or placement for a blind final test and keep its reference values out of the development materials.
- Stretch: Add a second sensor to quantify how non-uniform mixing changes the estimate. Do not present a ppm threshold alone as an air-change measurement.

DELIVERABLES

- Minimum prototype: A working sensor node with live CO2 readings and a ventilation-rate estimate plus an explicit uncertainty or unreliable state.
- A reference-versus-estimate table across at least three conditions, including the blind condition, and a comparison with the ppm-threshold baseline.
- A documented limits page covering sensor type, sensor placement, well-mixed-air assumptions and situations in which no reliable estimate should be shown.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 08 | SMART CITY & INFRASTRUCTURE

PS 34 / 35

BUILD TYPE | HARDWARE + SOFTWARE

PipeEcho: Tabletop Acoustic Leak-Localization Bench

PROBLEM STATEMENT

Small leaks in water distribution systems can be hard to locate when inspection points are limited. Build a safe tabletop test rig that uses acoustic or vibration observations to estimate which section of a model pipe network is leaking.

DESCRIPTION / IMPLEMENTATION

- Create a low-pressure tabletop pipe loop with controllable leak points and one or more contact microphones, piezo sensors or vibration sensors.
- Extract repeatable signal features and rank likely leak sections; compare the result with a simple amplitude-threshold baseline.
- Test changes in flow, sensor placement, background noise and multiple candidate leak points. Use only a contained bench rig, never a live utility network.

DELIVERABLES

- A physical pipe-loop demonstrator with sensor readings and selectable test leak points.
- A simple interface showing the suspected section, evidence and confidence or uncertainty.
- A test table reporting localization accuracy across sensor placements and noise conditions.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE

TRACK 09 | OPEN INNOVATION IN HARDWARE

PS 35 / 35

BUILD TYPE | HARDWARE + SOFTWARE OPTIONAL

Open Innovation in Hardware: Build a Physical Solution to a Validated Need

PROBLEM STATEMENT

Identify a real, observable problem that calls for a physical intervention and develop an original hardware-first solution. The problem may come from campus life, accessibility, resource use, safety or community operations. Choose a bounded use case and validate the need with evidence rather than assumptions.

DESCRIPTION / IMPLEMENTATION

- Define the intended users, setting and current failure; validate the need through observation, consent-based feedback or a documented representative scenario.
- Build a physical prototype using appropriate sensors, actuators, embedded control, materials or mechanisms. Software may support the prototype, but the physical system must be central to the solution.
- Test reliability, usability, safety, power or material constraints, cost and failure modes against a simple baseline. Clearly distinguish measured results from estimates.

DELIVERABLES

- A working hardware prototype with the key physical components, a bill of materials and assembly or fabrication notes.
- An end-to-end demonstration showing inputs, physical outputs and the intended use case; include source code, CAD files, circuit diagrams or mechanical drawings where applicable.
- A concise repeatable test report with measured results, known limitations and safety, repairability, privacy or cost considerations relevant to the prototype.

BUILD THE PROOF / TEST FAILURE CASES / SHOW YOUR EVIDENCE